Topic 1: Issue Report Classification

Pham Ngoc Hai

Abstract

The NLBSE'24 dataset contains 3,000 GitHub issue reports from five open-source repositories. Each issue is labelled as a bug, feature, or question. This project builds one classifier for each repository to predict these labels from the issue title and description. We compare five text models and one dummy baseline using five-fold cross-validation on the training set. Based on this comparison, we choose word-and-character TF–IDF with LinearSVC. On the official test set, our chosen model achieved a cross-repository macro-F1 of 0.754. We also examined three exact train/test overlaps; excluding them changed macro-F1 by only 0.000257.

1 Problem and Research Questions

The NLBSE'24 tool competition ask us to classify GitHub issues as bugs, features, or questions. Each model receive an issue title and description as input. It returns exactly one label. The project answer three research questions. These questions guides the model comparison and final evaluation. This task follows the NLBSE'24 tool competition [1] and earlier work on issue classification [3, 2].

The research questions are:

1. Which text configuration performs best under training-only cross-validation?
2. How well does the chosen model perform on the official test set, including differences across repositories and labels and comparison with the NLBSE'24 results?
3. Which errors does the chosen model make most often, and how can the approach be improved?

2 Dataset and Invariants

The NLBSE'24 tool competition provides 3,000 issues from five repositories. Each split contain 1,500 issues. The training set provide 100 examples for every repository and label combination. The test set remain fixed for final evaluation. It has the same balanced structure as the training set. No title is missing. Two test issues have a missing description. Table 1 summarizes these checks.

Table 1: Size and quality checks for the official dataset split.

Split	Rows	Rows/repo	Rows/repo-label	Null title	Null body	Dup. groups
train	1500	300	100	0	0	1
test	1500	300	100	0	2	0

3 Implementation Architecture

The four notebooks are the main execution path. Each notebook call functions from the project package; model logic is not duplicated inside the notebooks. This sequence make the complete experiment easier to follow.

1. Notebook 01 (*Data Inspection*) calls the package data-loading, text-preparation, and inspection functions. It displays balance, missing values, duplicates, and exact overlaps.
2. Notebook 02 (*Chosen Model*) displays the executed five-fold comparison. Its optional rerun calls `run_cross_validation`, which fits all six candidates using training data only.
3. Notebook 03 (*Final Evaluation*) applies the selected configuration separately to five repositories. Its optional rerun calls the final-evaluation workflow.
4. Notebook 04 (*Results*) reads the final metrics and predictions to display repository scores, class scores, the confusion matrix, and common errors.

For example, `build_model_text` prepares the issue text and `run_cross_validation` performs the train-only model comparison.

The executed outputs are stored as JSON, CSV, and PNG files. For example, `metrics.json` contains the final scores and `predictions.csv` contains row-level true and predicted labels. The final-result directory also contains confusion-matrix images. The notebooks read these files to display tables and figures.

4 Preprocessing, Data Inspection, and Protocol

The preprocessing step join the issue title and description into one text field. A missing description become an empty string. Repeated whitespace is reduced to one space. These steps preserves code, Markdown, URLs, identifiers, punctuation, and letter case. Repository name and creation time are not model features.

The function first validates the five required columns. It then returns a new Pandas series with the same row index as the input frame. It inserts `[TITLE]` and `[BODY]` markers so the vectorizer can distinguish the two fields. It does not lowercase, stem, remove stop words, or strip code. Figure 1 shows the exact construction and the main inspection operations.

preprocessing.py and dataset_inspection.py - prepare and inspect the text

```python
def build_model_text(frame: pd.DataFrame) -> pd.Series:
    validate_issue_frame(frame)
    texts = []
    for title, body in zip(frame['title'], frame['body']):
        body_text = '' if pd.isna(body) else body
        text = f'[TITLE] {title} [BODY] {body_text}'
        texts.append(normalize_whitespace(text))
    return pd.Series(texts, index=frame.index, name='text')

def _inspect_split(frame: pd.DataFrame) -> dict[str, Any]:
    model_text = build_model_text(frame)
    repo_labels = frame.groupby(['repo', 'label']).size()
    return {
        'row_count': len(frame),
        'missing_values': frame.isna().sum(),
        'text_length_statistics': _length_statistics(model_text),
        'exact_duplicate_groups': _duplicate_groups(frame),
    }
```

Figure 1: Text construction and per-split dataset inspection.

The inspection counts missing values and groups rows by repository and label. It calculates text-length statistics and finds exact duplicate groups. For the cross-split check, the code hashes

the raw title-description pair with SHA-256 after converting only a missing description to an empty string. It does not normalize whitespace before this comparison. This keeps the overlap rule strict and repeatable.

We also compare exact title-description pairs across the two splits. Three test issues also appear in training. One of these pairs has different labels. The main evaluation keeps all official test issues. A second evaluation removes only these three overlaps to measure their effect.

5 Chosen Model Selection

The comparison include five text models and one dummy baseline. The text models are two word TF–IDF logistic-regression configurations and three word-and-character TF–IDF LinearSVC configurations. The dummy baseline ignore the issue text and always predicts one class.

The word vectorizer uses one-word and two-word sequences. The character vectorizer uses three-to-five-character sequences inside word boundaries. Both use sublinear term frequency. Logistic regression is tested at $C = 1$ and $C = 4$. LinearSVC is tested at $C = 0.5$, $C = 1$, and $C = 2$. Figure 2 shows how the word and character matrices are joined before LinearSVC receives them.

model_selection.py - word and character TF-IDF feeding LinearSVC

```
def _word_vectorizer() -> TfidfVectorizer:
    return TfidfVectorizer(
        ngram_range=(1, 2), sublinear_tf=True
    )

def _char_vectorizer() -> TfidfVectorizer:
    return TfidfVectorizer(
        analyzer='char_wb', ngram_range=(3, 5),
        sublinear_tf=True,
    )

def _word_char_linear_svc(c_value: float) -> Pipeline:
    features = FeatureUnion([
        ('word', _word_vectorizer()),
        ('char_wb', _char_vectorizer()),
    ])
    return Pipeline([
        ('tfidf', features),
        ('classifier', LinearSVC(
            C=c_value, random_state=42, max_iter=5000)),
    ])
```

Figure 2: Construction of the word-and-character TF–IDF LinearSVC pipeline.

Five-fold cross-validation divides each repository's training issues into five parts. Each candidate use four parts for training and one part for validation. This process repeats until every part has been used for validation. The five folds gives one mean macro-F1 score for each repository and candidate. We then average the five repository scores. The official test set is not read during this comparison.

Each repository contains 300 training issues. One fold therefore fits on 240 rows and validates on 60 rows, with 20 validation rows for each label. The vectorizers and classifier are created again inside every fold, so validation text cannot influence the TF–IDF vocabulary. Six candidates, five repositories, and five folds produce 150 fitted pipelines. Of these, 125 are fits of the five text models and 25 are dummy fits. Figure 3 shows the loop.

model_selection.py - every candidate is refitted inside every training fold

```
train = store.load('train')  # the test split is not loaded here
model_text = build_model_text(train)
for repository in repositories:
    mask = train['repo'] == repository
    text = model_text.loc[mask]
    labels = train.loc[mask, 'label']
    splitter = StratifiedKFold(
        n_splits=5, shuffle=True, random_state=42
    )
    folds = list(splitter.split(text, labels))
    for spec in candidate_specs():
        for fold, (train_idx, valid_idx) in enumerate(folds, 1):
            estimator = spec.factory()
            estimator.fit(text.iloc[train_idx], labels.iloc[train_idx])
            predicted = estimator.predict(text.iloc[valid_idx])
            score = f1_score(
                labels.iloc[valid_idx], predicted,
                labels=('bug', 'feature', 'question'),
                average='macro', zero_division=0,
            )
            fold_records.append((spec.name, repository, fold, score))
```

Figure 3: Training-only five-fold comparison for every repository and candidate.

Every candidate produces 25 macro-F1 values: five folds for each of five repositories. The code first averages the five folds within each repository, then averages the five repository means. It selects the highest global mean. An exact tie is resolved by lower population standard deviation, lower complexity rank, and candidate name, in that order.

Word-and-character TF–IDF with LinearSVC at $C = 1$ achieved the highest mean cross-validation macro-F1, 0.748. We therefore selected this configuration for all five repositories. Table 2 reports every candidate, including the dummy baseline.

Table 2: Five-fold cross-validation results on the training set.

Model	Features / estimator	Mean macro-F1	Population std.
Dummy	None / Dummy	0.167	0.000
Word+char SVC C=0.5	Word+char TF-IDF / SVC	0.744	0.064
Word+char SVC C=1	Word+char TF-IDF / SVC	0.748	0.056
Word+char SVC C=2	Word+char TF-IDF / SVC	0.746	0.057
Word LR C=1	Word TF-IDF / LR	0.730	0.073
Word LR C=4	Word TF-IDF / LR	0.747	0.071

6 Applying the Chosen Model

The final stage read the selected candidate name from the saved cross-validation result. It checks that this name matches a defined candidate factory. It then loads only the training split. For each repository, it create a fresh copy of the chosen pipeline and fits it on all 300 training issues from that repository. This produces five separate models with the same parameters but different TF–IDF vocabularies and LinearSVC weights.

The five pipelines is saved as `.joblib` files. Their filenames, repositories, and SHA-256 hashes are recorded in the training manifest. The code verify that all five files exist and match their recorded hashes before loading the test split. Figure 4 shows the two repository loops.

workflow.py - fit five models, verify them, then route test rows by repository

```
for repository in sorted(ALLOWED_REPOSITORIES):
    mask = train['repo'] == repository
    estimator = selected_spec.factory()
    estimator.fit(train_text.loc[mask], train.loc[mask, 'label'])
    filename = repository.replace('/', '_') + '.joblib'
    joblib.dump(estimator, staged_models / filename)
    model_files.append({
        'repository': repository,
        'filename': filename,
        'sha256': _file_hash(staged_models / filename),
    })

models = _load_verified_models(bundle)
test = store.load('test')
test_text = build_model_text(test)
for repository in sorted(ALLOWED_REPOSITORIES):
    mask = test['repo'] == repository
    predicted = models[repository].predict(test_text.loc[mask])
    # Each issue is handled by the model trained for its repository.
```

Figure 4: Training five repository models and applying each model to its test rows.

The official test split is loaded once after training and verification. Each repository model receives the 300 test issues from the same repository. The primary policy keeps all 1,500 predictions. The leakage-sensitive policy keeps 1,497 predictions after removing the three exact overlaps. Each stored row contains the repository, true label, predicted label, text hash, and overlap flag. No test label is passed to `fit()` or used to change the pipeline.

7 Final Chosen-Model Evaluation

Our chosen model is trained once on all training issues from each repository. It achieves a cross-repository macro-F1 of 0.754385 on the official test set. The score is the arithmetic mean of the five repository macro-F1 scores. Tables 3 and 4 show where performance varies by repository and label.

Table 3: Repository-level macro metrics for our chosen model.

Policy	Repository	Macro precision	Macro recall	Macro F1
Primary	bitcoin/bitcoin	0.710	0.707	0.701
Leakage-sensitive	bitcoin/bitcoin	0.713	0.709	0.703
Primary	facebook/react	0.813	0.810	0.807
Leakage-sensitive	facebook/react	0.813	0.810	0.807
Primary	microsoft/vscode	0.731	0.727	0.727
Leakage-sensitive	microsoft/vscode	0.731	0.727	0.727
Primary	opencv/opencv	0.783	0.783	0.783
Leakage-sensitive	opencv/opencv	0.783	0.783	0.783
Primary	tensorflow/tensorflow	0.760	0.753	0.754
Leakage-sensitive	tensorflow/tensorflow	0.759	0.753	0.753

Table 4: Global per-class metrics for both evaluation policies.

Policy	Label	Precision	Recall	F1	Support
Primary	bug	0.796	0.754	0.771	500
Primary	feature	0.762	0.810	0.782	500
Primary	question	0.720	0.704	0.710	500
Leakage-sensitive	bug	0.798	0.754	0.771	499
Leakage-sensitive	feature	0.762	0.812	0.782	498
Leakage-sensitive	question	0.720	0.704	0.710	500

Table 5: Primary and leakage-sensitive policy summary.

Policy	Included rows	Excluded rows	Cross-repository macro-F1	Delta vs. primary
Primary	1500	0	0.754385	0.000000
Leakage-sensitive	1497	3	0.754642	0.000257

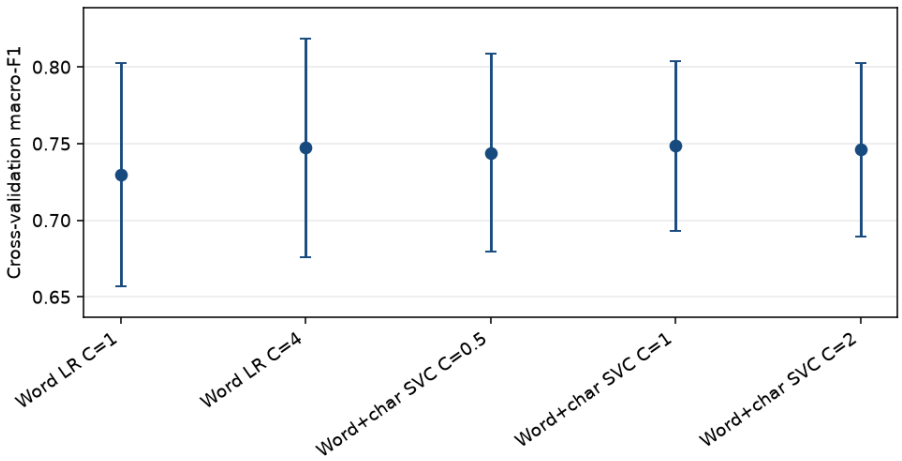

Figure 5: Zoomed comparison of the five text models. Error bars show population standard deviations; the dummy remains in Table 2.

8 Comparison with NLBSE'24 Results

The NLBSE'24 tool competition reports SetFit scores of 0.827 (headline) and 0.824 (detailed). Their RoBERTa and fastText scores are 0.792 and 0.718. Our chosen model achieved 0.754 on the official test set.

Table 6: Our results compared with the NLBSE'24 results.

System	Overall macro-F1
Our chosen model	0.754
NLBSE'24 detailed SetFit score	0.824
NLBSE'24 RoBERTa score	0.792
NLBSE'24 fastText score	0.718
NLBSE'24 headline SetFit score	0.827

9 Error Analysis

The largest error occur when questions are predicted as features. The next largest error is predicting bugs as questions. Table 7 shows the ten most frequent repository-specific errors. Figure 6 shows all predictions across the five repositories.

Table 7: Most frequent primary-policy confusion pairs.

Repository	True	Predicted	Errors
bitcoin/bitcoin	bug	question	34
facebook/react	question	feature	30
bitcoin/bitcoin	question	feature	23
microsoft/vscode	question	feature	23
tensorflow/tensorflow	question	bug	21
tensorflow/tensorflow	feature	question	20
microsoft/vscode	bug	question	18
microsoft/vscode	feature	bug	17
opencv/opencv	question	bug	17
opencv/opencv	bug	feature	16

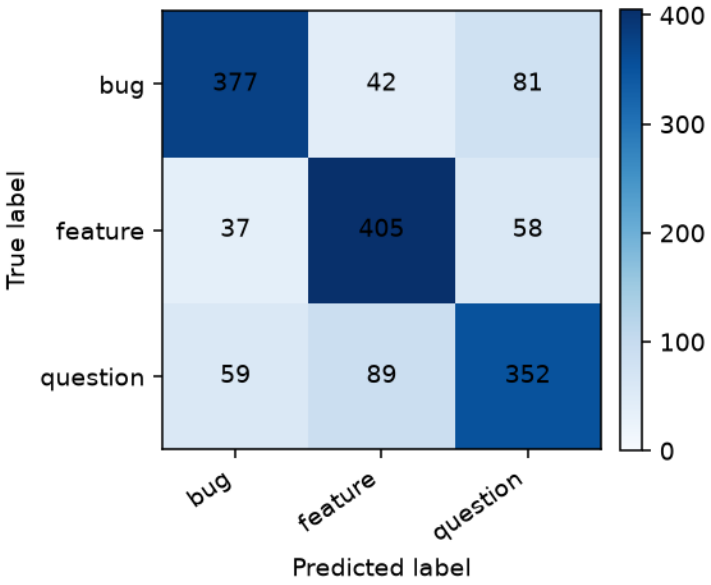

Figure 6: Aggregate primary-policy confusion matrix generated from final metrics.

10 Analysis and Improvements

The error table shows two main problems. The model confuses questions with features because both labels contain requests for a change or a new capability. It also confuses bugs with questions when an issue describes a failure and asks for an explanation at the same time. Word and character features capture terms such as “error”, “support”, and “request”, but they do not reliably capture the purpose of the sentence.

Performance also changes by repository. Bitcoin is the weakest repository with macro-F1 of 0.701. Its issue language is more technical and its reports contain shorter context. React is stronger at 0.807, where issue descriptions provide more repeated vocabulary. This suggests that repository conventions affect the task even when every repository has the same label counts.

These error patterns shows where the current features are insufficient.

The chosen model is one global configuration. It is simple and fast, but it cannot adapt its vocabulary or decision boundary to each repository. The main improvements are repository-specific tuning, more careful treatment of short titles, and error-driven review of question–feature

and bug–question pairs. Character features can be expanded for identifiers and code fragments. A sentence-embedding model can be tested later if a stronger semantic comparison is required, but it is outside the chosen-model result reported here. The proposed improvements focuses on the observed confusion pairs.

The three exact train/test overlaps have little effect. Removing them changes macro-F1 from 0.754385 to 0.754642, a difference of 0.000257. The reported result therefore does not depend materially on these rows.

11 Run Instructions and Conclusion

The four numbered notebooks are the recommended execution path. For local use, create a Python 3.11 environment and call its interpreter directly. PowerShell environment activation is not required. The same interpreter installs the dependencies and starts JupyterLab:

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m jupyter lab
```

After JupyterLab opens, run Notebooks 01, 02, 03, and 04 in numerical order. By default, model training is off. Notebook 02 sets `RERUN_CROSS_VALIDATION=False`, and Notebook 03 sets `RERUN_FINAL_EVALUATION=False`. With these defaults, the notebooks load and display the saved results. Set the relevant switch to `True` only when the corresponding training stage must be repeated.

The same notebook workflow also runs in Colab. Upload the complete project folder to Google Drive, open the notebooks from `notebooks/` in numerical order, and select *Runtime* > *Run all*. The README lists optional command-line checks, but they are not required for the notebook presentation workflow.

Our chosen model achieved 0.754. The NLBSE'24 tool competition reports 0.827 for SetFit. The three exact train/test overlaps changed the chosen-model result by only 0.000257. The experiment selected one configuration, measured its test performance, and identified its main errors and possible improvements. These results answer the three research questions without changing the official data split.

References

[1] Rafael Kallis, Giuseppe Colavito, Ali Al-Kaswan, Luca Pascarella, Oscar Chaparro, and Pooja Rani. The NLBSE'24 tool competition. In *Proceedings of the 3rd International Workshop on Natural Language-based Software Engineering*, 2024.

[2] Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. Ticket tagger: Machine learning driven issue classification. In *2019 IEEE International Conference on Software Maintenance and Evolution*, pages 406–409, 2019.

[3] Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. Predicting issue types on GitHub. *Science of Computer Programming*, 205:102598, 2021.